VI. LÝ THUYẾT LINUX

1. Process Lifecycle

Process là một chương trình đang được thực thi. Trong Linux, process có vòng đời từ lúc được tạo ra, thực thi cho đến khi kết thúc.

Một process thường trải qua các trạng thái:

Create → Ready → Running → Waiting → Terminated

Các khái niệm quan trọng trong process lifecycle gồm fork, exec, wait và zombie process.

1.1. fork()

fork() được sử dụng để tạo một process mới từ process hiện tại.

Process tạo ra process mới được gọi là **parent process**, process mới được gọi là **child process**.

Ví dụ:

```
pid_t pid = fork();


if (pid == 0) {
    printf("This is child process\n");
} else {
    printf("This is parent process\n");
}
```

Sau khi fork(), hai process có không gian thực thi riêng và có PID khác nhau.

Có thể kiểm tra PID bằng:

```
ps
```

Hoặc:

```
echo $$
```

fork() thường được sử dụng để tạo process mới trước khi process con thực hiện một chương trình khác.

1.2. exec()

exec() được sử dụng để thay thế chương trình hiện tại bằng một chương trình khác.

Điểm quan trọng là exec() **không tạo process mới**. Nó thay thế nội dung chương trình đang chạy trong process hiện tại.

Mô hình:

Child Process

|

exec()

|

v

New Program

Ví dụ:

execl("/bin/ls", "ls", "-la", NULL);

Process hiện tại sẽ thực thi chương trình ls.

Ví dụ khi shell chạy một command:

ls -la

Shell có thể tạo child process bằng fork() rồi sử dụng exec() để chạy chương trình ls.

1.3. wait()

wait() được parent process sử dụng để chờ child process kết thúc.

Ví dụ:

```
pid_t pid = fork();


if (pid == 0) {
    printf("Child running...\n");
    exit(0);
} else {
    wait(NULL);
    printf("Child finished\n");
}
```

wait() giúp parent thu nhận trạng thái kết thúc của child và tránh tạo zombie process.

1.4. Zombie Process

Zombie process là process **đã kết thúc nhưng entry của nó vẫn còn trong process table** vì parent chưa gọi wait() để thu nhận trạng thái kết thúc.

Một zombie process không còn thực thi chương trình nhưng vẫn giữ một số thông tin như PID trong process table.

Có thể kiểm tra bằng:

ps aux

Hoặc:

ps -el

Zombie thường được hiển thị với trạng thái:

Z

Nguyên nhân phổ biến:

- Child đã kết thúc.
- Parent vẫn đang chạy.
- Parent chưa gọi wait() hoặc waitpid().

Giải pháp là parent cần gọi:

wait(NULL);

hoặc:

waitpid(child_pid, NULL, 0);

2. File Permissions

Linux sử dụng permission để kiểm soát quyền truy cập vào file và directory.

Có ba nhóm người dùng:

User → Owner của file

Group → Nhóm người dùng

Other → Những người dùng khác

Ba quyền cơ bản:

r → Read

w → Write

x → Execute

Ví dụ:

-rwxr-xr--

Có thể phân tích:

- | rwx | r-x | r--

| | |

| | +---- Other

| +---------- Group

+---------- User

Giá trị:

Permission	Giá trị
r	4
w	2
x	1

Ví dụ:

rwx = 4 + 2 + 1 = 7

r-x = 4 + 0 + 1 = 5

r-- = 4 + 0 + 0 = 4

Vì vậy:

rwxr-xr-- = 754

Có thể kiểm tra permission bằng:

ls -l

Ví dụ:

-rwxr-xr-- 1 duy91 developers 1200 hello.sh

Trong đó:

duy91 → User/Owner

developers → Group

Thay đổi permission:

chmod 755 hello.sh

Thay đổi owner:

sudo chown duy91:developers hello.sh

3. I/O Redirection

Linux sử dụng các stream chuẩn để giao tiếp với chương trình:

stdin

stdout

stderr

3.1. stdin

stdin là **standard input**, thường nhận dữ liệu từ bàn phím.

Ví dụ:

read name

Người dùng nhập dữ liệu từ terminal và chương trình nhận dữ liệu thông qua stdin.

3.2. stdout

stdout là **standard output**, thường hiển thị kết quả trên terminal.

Ví dụ:

echo "Hello Linux"

Kết quả được đưa ra stdout.

Có thể redirect output vào file:

echo "Hello Linux" > output.txt

Lúc này nội dung không hiển thị trực tiếp trên terminal mà được ghi vào output.txt.

> sẽ ghi đè file.

Nếu muốn append:

echo "Hello Linux" >> output.txt

3.3. stderr

stderr là **standard error**, được sử dụng để xuất thông báo lỗi.

Ví dụ:

ls /abc

Nếu /abc không tồn tại, thông báo lỗi được gửi đến stderr.

Có thể redirect stderr:

ls /abc 2> error.log

Trong đó:

0 → stdin

1 → stdout

2 → stderr

Có thể redirect cả stdout và stderr:

command > output.log 2> error.log

Hoặc:

command > all.log 2>&1

4. Pipes

Pipe | cho phép truyền stdout của command này thành stdin của command tiếp theo.

Ví dụ:

ps aux | grep java

Luồng dữ liệu:

ps aux

  |

  | stdout

  v

 grep java

  |

  v

 Output

Một ví dụ khác:

cat app.log | grep ERROR

Có thể kết hợp nhiều command:

cat app.log | grep ERROR | sort | uniq -c

Luồng xử lý:

app.log

  ↓

grep ERROR

  ↓

sort

↓

uniq -c

↓

Kết quả

Pipe đặc biệt hữu ích khi xử lý log hoặc dữ liệu text.

5. System Logs

Linux lưu lại nhiều thông tin hoạt động của hệ thống dưới dạng log.

Log có thể giúp:

- Kiểm tra lỗi.
- Theo dõi service.
- Kiểm tra authentication.
- Debug ứng dụng.
- Phân tích sự cố hệ thống.

Một thư mục quan trọng là:

/var/log

5.1. /var/log

Kiểm tra:

ls -lah /var/log

Một số log thường gặp:

/var/log/syslog

/var/log/auth.log

/var/log/kern.log

Có thể xem log:

cat /var/log/syslog

Hoặc:

tail /var/log/syslog

Xem realtime:

tail -f /var/log/syslog

tail -f rất hữu ích khi theo dõi log của service đang chạy.

6. syslog

syslog là cơ chế chuẩn được Linux sử dụng để ghi và quản lý system log.

Các chương trình và service có thể gửi log đến hệ thống logging.

Có thể tìm thông tin liên quan đến một service:

grep "ssh" /var/log/syslog

Hoặc tìm lỗi:

grep -i "error" /var/log/syslog

Syslog đặc biệt hữu ích khi cần kiểm tra hoạt động của hệ thống và tìm nguyên nhân khi xảy ra lỗi.

7. journalctl

Trên các hệ thống sử dụng systemd, journalctl được sử dụng để xem system journal.

Xem toàn bộ journal:

journalctl

Xem log mới nhất:

journalctl -n 50

Theo dõi log realtime:

journalctl -f

Xem log của một service:

journalctl -u ssh

Xem log của service trong khoảng thời gian:

journalctl -u ssh --since "1 hour ago"

Xem các message có mức độ lỗi:

journalctl -p err